Raízes do Nordeste — relatório técnico de estudo

1. Introdução e objetivos

A rede de lanchonetes precisa integrar unidades com cardápio e estoque próprios, atendendo APP, TOTEM, BALCAO, PICKUP e WEB. Esta implementação acadêmica oferece uma API REST persistente que centraliza pedidos, pagamento simulado, operação da cozinha e fidelização. O canal de origem é obrigatório e consultável, permitindo acompanhamento operacional. O objetivo é fechar o fluxo pedido → pagamento mock → preparo → entrega, com validação de estoque e controle de acesso.

Documento de referência: UNINTER_Roteiro_Back_End_Copia_Adaptada_Estudo.pdf (2026), cópia adaptada para estudo, fornecida pelo usuário. Antes da entrega institucional, conferir o roteiro oficial, preencher identificação acadêmica e adequar a formatação exigida. O relatório registra a implementação técnica; não comprova publicação no GitHub nem substitui revisão pelo estudante.

2. Análise e requisitos

ID	Requisito	Prioridade e evidência
RF01	Cadastro e autenticação por token; autorização por perfil	Essencial; /usuarios, /auth/login; hash BCrypt e JWT
RF02	Gestão da rede/unidades	Essencial; cadastro/atualização ADMIN; gerente atualiza sua unidade
RF03	Catálogo global e cardápio por unidade	Essencial; produtos, preços em centavos e disponibilidade
RF04	Criação e consulta de pedidos com itens e canalPedido	Essencial; canal obrigatório e filtro por canal/status
RF05	Atualização de status e cancelamento	Essencial; máquina de estados com autorização operacional
RF06	Estoque por unidade e movimentos	Essencial; operações atômicas e proibição de saldo negativo
RF07	Pagamento externo simulado e registro de retorno	Essencial; gateway desacoplado, aprovação, negativa, erro e estorno mock
RF08	Fidelidade com consentimento e resgate	Implementado; 1 ponto por real efetivamente pago na entrega; resgate de 100 pontos = R$ 10
RF09	Promoção/campanha	Implementado; cupom NORDESTE10: 10% sobre subtotal mínimo de R$ 50, sem vigência dinâmica
RF10	Privacidade e auditoria	Implementado; leitura de dados próprios, retirada de consentimento e desidentificação

ID	Requisito não funcional	Decisão e limite
RNF01	Segurança	JWT HS256 com expiração, hash BCrypt, perfis e vínculo com unidade; HTTPS deve ser configurado no deploy
RNF02	Integridade	FK/CHECK/UNIQUE e transação Spring JDBC com bloqueios FOR UPDATE; estoque, pontos e pedido confirmam juntos
RNF03	Rastreabilidade	Auditoria de operações sensíveis e leituras; requestId por resposta; sem senha/token nos registros

RNF04	Documentação	Swagger /docs, OpenAPI exportado e contratos de entrada/saída tipados
RNF05	Desempenho	Paginação até 100 registros e índices de busca; testes de concorrência; sem alegar benchmark de produção
RNF06	Disponibilidade	Healthcheck com consulta real ao banco; H2 local não oferece alta disponibilidade
RNF07	Tolerância a falhas	Mock indisponível gera registro ERRO; nova chave permite nova tentativa; não autoriza cozinha antes de aprovação
RNF08	Reprodutibilidade	Java 21, versões Maven fixadas e checksums estritos, migrations versionadas, seed idempotente e testes isolados

Escolheu-se H2 para dispensar um serviço externo no ambiente de estudo. Locks transacionais por recurso protegem estoque e fidelidade; essa decisão limita a concorrência de uma rede em produção. Uma evolução real exige PostgreSQL, controle de concorrência, filas, backups, monitoramento e medição de carga. A integração mock foi escolhida por exigência do roteiro e não coleta dados de cartão.

3. Modelagem e arquitetura

Os diagramas estão em docs/diagramas/: casos de uso, DER completo, classes conceituais e arquitetura em SVG/PNG, com fontes DOT. A sequência do fluxo crítico está também em docs/sequencia.md.

A API valida DTOs Java com Bean Validation e aplica autenticação. Application orquestra transações e gateway. Domain define enums, transições, preços e regras de resgate/cupom. Infrastructure contém H2, migrations Flyway SQL, hashing, JWT e gateway simulado. O domínio é implementado com funções e registros persistentes; o diagrama de classes é uma visão conceitual dessas entidades, não uma alegação de ORM ou DDD completo.

Uma unidade possui vários produtos em estoque, e cada produto pode estar em várias unidades; stocks usa PK composta (unidade_id, produto_id). Usuário possui pedidos, movimentos de fidelidade e auditoria. Pedido pertence a uma unidade e possui itens e tentativas de pagamento independentes. Itens guardam o nome/preço histórico do produto. Chaves de idempotência são únicas por ator e operação. A migration V1__initial.sql define todas as tabelas, PK, FK, CHECK e índices.

Feature crítica: realizar pedido e solicitar pagamento

Atores: Cliente (APP/WEB/TOTEM/PICKUP), Atendente (BALCAO), Cozinha, Gerente/Administrador e Gateway mock.

Pré-condições: usuário ativo autenticado; unidade ativa; produtos ativos, ofertados e disponíveis; cliente válido; chave Idempotency-Key; canal válido. Atendente/gerente informa clienteId e só atua na unidade vinculada. Cliente atua em seus próprios pedidos.

Fluxo principal:
1. A API valida canalPedido, itens e quantidades. O servidor calcula preços e descontos, sem confiar em valores monetários enviados pelo cliente.

2. Uma transação verifica estoque e consentimento/saldo para resgate; cria pedido e itens; registra saídas de estoque e eventual débito de pontos; grava auditoria e chave de idempotência.
3. Pedido inicia em AGUARDANDO_PAGAMENTO. O cliente solicita pagamento mock com chave de idempotência própria.
4. O gateway simulado responde com status e payload. A aplicação persiste a tentativa; apenas APROVADO altera o pedido para EM_PREPARO.
5. Cozinha ou atendimento marca PRONTO. Atendimento/gerente/admin marca ENTREGUE.
6. Na entrega, cliente com consentimento ativo recebe totalCentavos // 100 pontos; reentrega não pode creditar novamente.

Pós-condições: pedido e canal consultáveis, estoque atualizado, pagamento registrado e auditoria persistida. Saldo de pontos só aumenta após entrega.

Exceções: 401 sem autenticação; 403 perfil/unidade/cliente incompatível; 404 recurso inexistente; 422 canal ausente/inválido ou itens repetidos; 409 estoque/pontos insuficientes, cupom inválido, transição proibida ou chave reutilizada com corpo diferente. NEGADO e ERRO são resultados persistidos do gateway (HTTP 201 porque a tentativa foi criada), sem avanço do status. Nova tentativa usa nova chave. Reenvio com mesma chave/corpo devolve o recurso existente no estado atual, sem repetir efeitos.

Cancelamento antes da entrega devolve estoque exatamente uma vez; pagamento aprovado recebe ESTORNADO mock. Cliente pode cancelar apenas enquanto aguarda pagamento; equipe de atendimento autorizada pode cancelar EM_PREPARO/PRONTO. Cozinha pode somente marcar PRONTO. Pedido entregue não pode ser cancelado no MVP. Pontos resgatados voltam no cancelamento se o consentimento ainda estiver ativo. Revogação de consentimento zera o saldo e impede novos créditos e devoluções enquanto revogado; o benefício já aplicado ao pedido não é removido.

Pedidos não pagos mantêm reserva de estoque até cancelamento manual. Expiração automática, reembolso real, compra anônima em totem, login refresh/logout com revogação individual e campanhas dinâmicas são evoluções propostas; não foram implementadas.

4. API e contratos

Consulte docs/api.md e docs/openapi.json; na API em execução, /docs e /openapi.json. Cada endpoint tem parâmetros, contratos tipados, permissões descritas e exemplos. Operações administrativas usam cadastro de cliente seguido de atribuição de perfil pelo ADMIN. Não há elevação de perfil pelo cadastro público.

Padrão de erro: {error, message, details:[{field,issue}], timestamp, path, requestId}. Detalhes de validação omitem valores de entrada para não repetir senhas. Listagens retornam {items,page,limit}. Dinheiro usa centavos inteiros, evitando erro de ponto flutuante. IDs históricos são preservados; recursos são desativados com ativo=false, sem apagar vendas.

Status: AGUARDANDO_PAGAMENTO → EM_PREPARO (apenas pagamento aprovado) → PRONTO → ENTREGUE. CANCELADO é terminal. Não há salto direto de AGUARDANDO_PAGAMENTO para PRONTO/ENTREGUE.

5. LGPD, privacidade, segurança e auditoria

Dados coletados: nome, email e hash de senha para identificação/autenticação; identificador interno, itens, unidade, canal, preços e datas para executar e acompanhar pedidos; perfil e unidade de funcionários para autorização; consentimento (estado, versão 1.0, instante) e movimentos de pontos para fidelização. Não são coletados CPF, telefone, endereço residencial ou dados de cartão. O endereço da unidade é comercial/fictício no seed.

Finalidade e bases legais propostas: execução de contrato para pedidos e conta; consentimento específico facultativo para fidelidade; logs estritamente necessários à segurança devem ter base e retenção revisadas pelo controlador. A descrição técnica não constitui parecer jurídico. A fidelidade é opcional e consentimento pode ser retirado por PUT /fidelidade/consentimento. O saldo é zerado em retirada, com movimento de histórico e evento de auditoria. A versão e o instante do consentimento/revogação são persistidos; o histórico de alterações consta na auditoria.

Senhas: BCrypt com salt aleatório por usuário (custo 12); comparação em tempo constante; jamais retornadas. JWT: HS256, sub/iat/exp/iss/aud obrigatórios, expiração padrão de 3600 segundos e segredo aleatório local de 256 bits. O perfil ativo é consultado no banco a cada requisição, portanto alteração de perfil e desativação se aplicam a tokens já emitidos. A chave privada .env tem permissões 0600 e está ignorada pelo Git.

Autorização: clientes acessam somente recursos próprios; empregados só recursos da unidade atribuída; ADMIN administra a rede. Logs de leitura de perfil/pedidos/pagamentos/fidelidade/auditoria e eventos de criação/status/estoque/consentimento usam apenas IDs e ação, sem credenciais, email, corpo de requisição ou dados de cartão. O endpoint de auditoria é exclusivo do ADMIN.

DELETE /usuarios/me exige perfil CLIENTE e nenhum pedido em aberto. Nome/email/hash são substituídos, consentimento retirado, saldo zerado e conta desativada; tokens antigos deixam de funcionar. IDs e histórico operacional permanecem: trata-se de desidentificação/pseudonimização local, e não garantia de anonimização irreversível em sistemas externos.

Estratégia de retenção proposta: revisar periodicamente contas inativas e necessidade de dados; propor 180 dias para auditoria de desenvolvimento, ajustados à finalidade e obrigações aplicáveis; manter registros de venda pelo prazo definido pelo controlador. Limpeza por prazo não é automática no MVP. Exportação pessoal, gestão de solicitações LGPD e descarte de backups exigem processo operacional futuro.

Antes de produção: definir política de retenção aplicável, HTTPS, armazenamento central de segredos, proteção contra força bruta/rate limiting, controle de backup/restauração e segregação de dados. O presente ambiente é destinado ao estudo e não valida capacidade/segurança de operação pública.

6. Plano de testes e evidências

A coleção Postman possui 20 cenários ordenados, positivos e negativos, com testes de status e

conteúdo. docs/plano-testes.md apresenta entrada, pré-condições e resultado esperado de cada cenário; docs/evidencias-colecao-java.json é o resumo sanitizado da execução real em Java via JUnit e HttpClient. A suíte JUnit 5 possui 32 testes em banco temporário, incluindo concorrência de estoque/idempotência, isolamento entre clientes/unidades, pagamento negado/indisponível, consentimento, anonimização e repetição de migrations/seed. docs/evidencias-junit-api.xml e docs/evidencias-junit-colecao.xml registra a execução atual.

Os resultados são evidência da máquina atual e do código correspondente; novas alterações exigem nova execução. Importar a coleção não prova aprovação dos cenários. Contagem de testes, falhas e data real constam nos arquivos de evidência. Testes não publicam o ambiente nem validam restauração de snapshot em uma nova tarefa.

7. Entrega técnica e conclusão

Código Java, migrations Flyway, scripts de JVM/Maven, pom.xml, seed, .env.example, README, OpenAPI, coleção, plano de testes e diagramas estão no checkout. O PDF técnico é gerado a partir deste relatório e dos diagramas. Repositório pretendido: https://github.com/jefersonrivaldo/raizes-do-nordeste ; o código e os documentos foram enviados à branch main após autorização do proprietário. As URLs de download sem login retornaram HTTP 404 nesta sessão; a visibilidade pública ainda precisa ser verificada. Acesso local ao Swagger usa a rota /docs com API iniciada; não foi realizado deploy público.

O fluxo crítico é implementado com persistência real e operações transacionais. As limitações são explicitadas para separar capacidades demonstradas de evoluções propostas. Para entrega acadêmica, o estudante deve revisar o código/relatório, inserir seus dados de identificação, verificar a versão oficial do roteiro, verificar a visibilidade do repositório e testar os links pelo acesso do avaliador. Histórico mínimo de cinco commits é uma recomendação do roteiro; não foi fabricado nesta implementação.

Referências

UNINTER. Projeto Multidisciplinar, trilha Back-end: rede Raízes do Nordeste. Cópia adaptada para estudo, 2026, fornecida pelo usuário; documento oficial deve ser consultado.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018 (LGPD). https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm . Referência normativa indicada, sem alegar consulta jurídica.

SPRING. Documentação oficial do Spring Boot. https://docs.spring.io/spring-boot/ . Referência técnica para aprofundamento.

H2. Documentação do banco H2. https://h2database.com/html/main.html . Referência técnica para aprofundamento.

OWASP. Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html . Referência para revisão de configuração em produção.

Diagrama — casos-de-uso

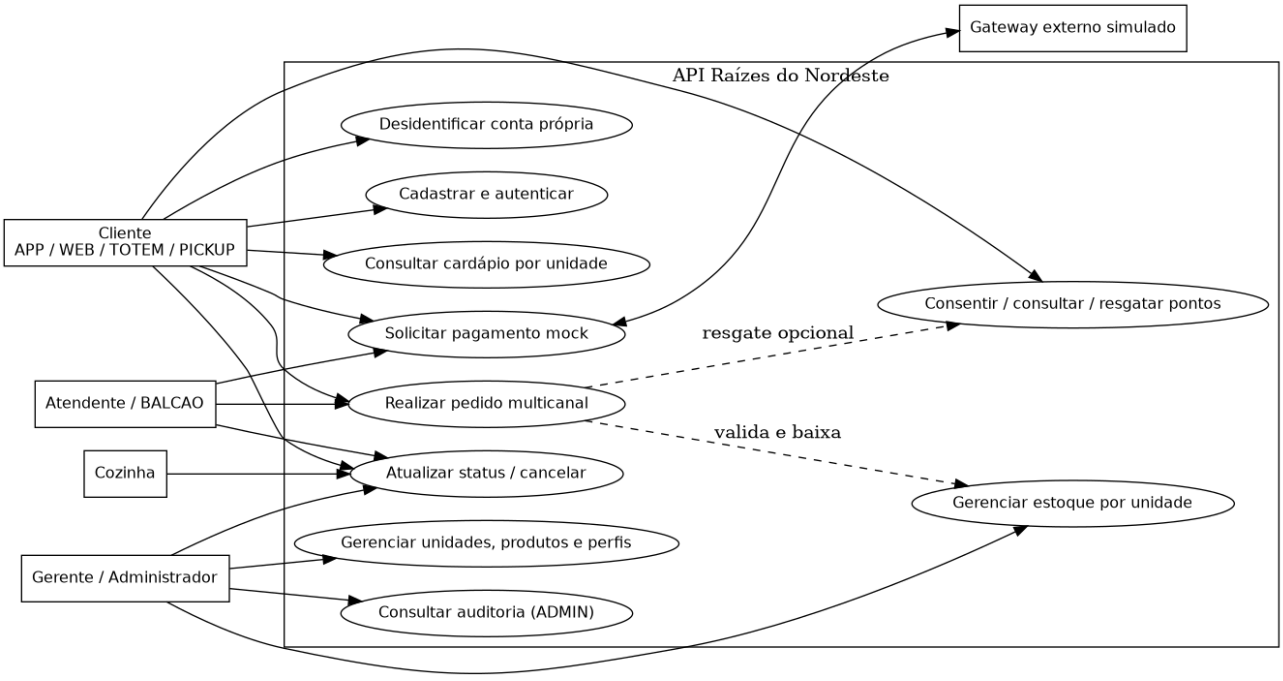


Diagrama — classes

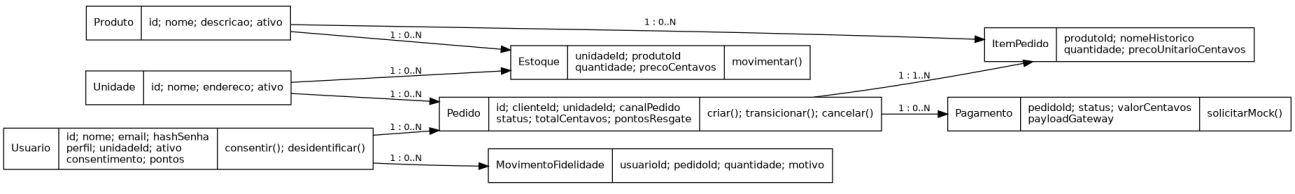


Diagrama — arquitetura

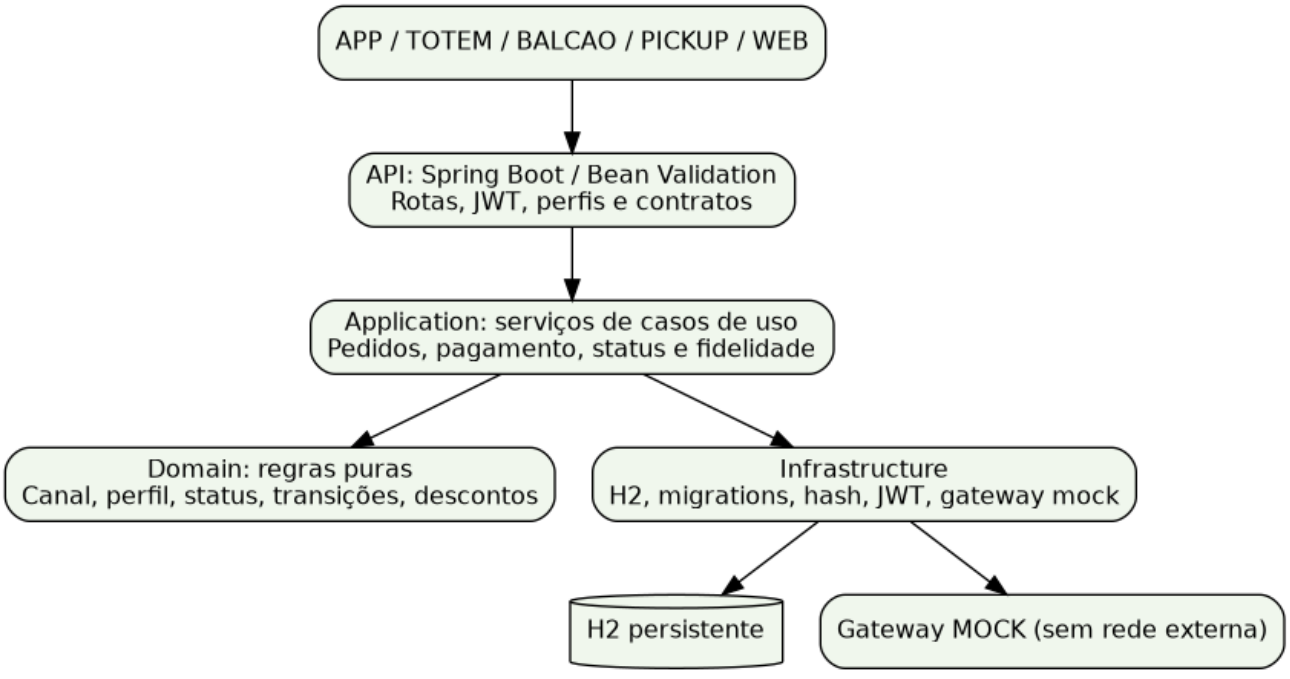


Diagrama — der

